

Скриптинг на пальцах

Как в наш движок встроили язык Lua, как на нём сделали игру «защити ядро от волн» и как устроены пять допфич.

Объяснение простыми словами, шаг за шагом, у каждого шага указаны файлы с кодом.

Если надо объяснить за 30 секунд

Движок написан на C++, и любая правка в C++ требует пересборки. Поэтому правила игры мы вынесли в маленький язык **Lua**: файлы `.lua` можно менять прямо во время игры.

Внутри движка живёт одна «Lua-машина» (класс `ScriptSystem`). Она читает файлы и раз в кадр вызывает у каждого объекта функцию `on_update`. Скрипт управляет миром через выданный ему «пульт» — набор функций движка вроде `get_position` или `spawn_prefab`. Объект он держит не напрямую, а через «номерок» (хэндл). Поэтому удалённый объект не роняет движок.

Сверху мы добавили пять допфич:

- движок сам замечает сохранённый файл: проверка идёт в фоне через job system;
- код подменяется прямо в Play, состояние игры при этом можно сохранить;
- зависший или жадный скрипт останавливается по лимитам;
- скрипты видно в профайлере Trasy;
- VS Code подсказывает функции движка.

Как читать этот файл

Простыми словами

Синий блок — суть на бытовом примере.

Где в коде

Серый блок — файл и функция, где это сделано. Открывай и смотри.

Если спросят

Жёлтый блок — короткие ответы на вопросы, которые вероятны на защите.

Все пути — от корня репозитория. Основной файл всей подсистемы — `src/scripting/ScriptSystem.cpp` (650 строк).

Что внутри

0 · Словарик: 15 слов, без которых непонятно
1 · Большая картина: что просили и что сделали
2 · Шаг 1. Положили Lua внутрь движка
3 · Шаг 2. Одна «Lua-машина» на весь редактор
4 · Шаг 3. Пульт от движка для скрипта
5 · Шаг 4. Номерок вместо куртки (хэндлы)
6 · Шаг 5. C++-база и Lua-наследник
7 · Шаг 6. Как файл .lua становится классом
8 · Шаг 7. Что происходит на Play и Stop
9 · Шаг 8. Один кадр игры
10 · Шаг 9. Префабы — шаблоны в JSON
11 · Шаг 10. Поля и инспектор
12 · Шаг 11. Сама игра на Lua

13 · Шаг 12. Ошибки не роняют движок
14 · Допфича 1. Движок сам замечает сохранение
15 · Допфича 2. Подмена кода прямо в игре (L1/L2)
16 · Допфича 3. Лимиты: время и память
17 · Допфича 4. Скрипты в профайлере Tracy
18 · Допфича 5. Подсказки в VS Code
19 · Тесты: чем доказано, что всё работает
20 · Три сценария целиком: от нажатия до результата
21 · Вопросы на защите: короткие ответы
22 · Шпаргалка: что где лежит

0 · Словарик

Lua	Маленький язык программирования. Его код лежит в файлах <code>.lua</code> , и чтобы его поменять, движок пересобирать не нужно.
Lua state	«Lua-машина»: место в памяти, где хранится и выполняется весь Lua-код. У нас она одна на весь редактор.
sol2	Библиотека-переводчик между C++ и Lua. Без неё пришлось бы вручную перекладывать каждое число между двумя языками.
Биндинг	Функция движка, которую мы «выдали» скрипту. Например, <code>get_position</code> : Lua зовёт её, а выполняется C++.
Сущность (Entity)	Объект в мире: ядро, враг, пол. Сама по себе это просто номер.
Компонент	Кусок данных у сущности: позиция (<code>Transform</code>), модель, тег, скрипт (<code>ScriptComponent</code>).
Класс поведения	Таблица в Lua с функциями <code>on_create</code> , <code>on_update</code> , <code>on_destroy</code> . У нас их три: <code>Waves</code> , <code>Enemy</code> и <code>Core</code> .
Экземпляр	Живая копия класса для конкретной сущности во время игры. У каждого врага свой экземпляр и свои переменные <code>self.*</code> .
Хэндл	«Номерок из гардероба»: скрипт хранит не сам объект, а номер и проверку, что объект ещё жив.

Префаб	Шаблон объекта в файле JSON: какие компоненты и какие числа. По нему одной командой создаётся враг.
Поле (field)	Число, строка или флаг поведения, видимые в инспекторе: <code>speed</code> , <code>health</code> , <code>damage</code> .
Hot reload	Подмена кода на лету: сохранил файл, и игра уже работает по-новому, без перезапуска.
Job system	Пул рабочих потоков из ЛР 1. Туда отдают тяжёлую работу, чтобы кадр не тормозил.
Главный поток / воркер	Главный поток рисует кадры и управляет игрой. Воркер — фоновый помощник из job system.
Edit / Play / Stop	Режимы редактора. В Edit сцену настраивают, в Play идёт игра, Stop возвращает сцену как было до Play.

1 · Большая картина

Что просило ТЗ и что получилось в итоге.

ПРОСТЫМИ СЛОВАМИ

Представь театр. Сцена, свет и звук — это **движок на C++**: он рисует, считает физику, проигрывает анимацию. А **сценарий** — кто куда идёт и что делает — написан на **Lua**.

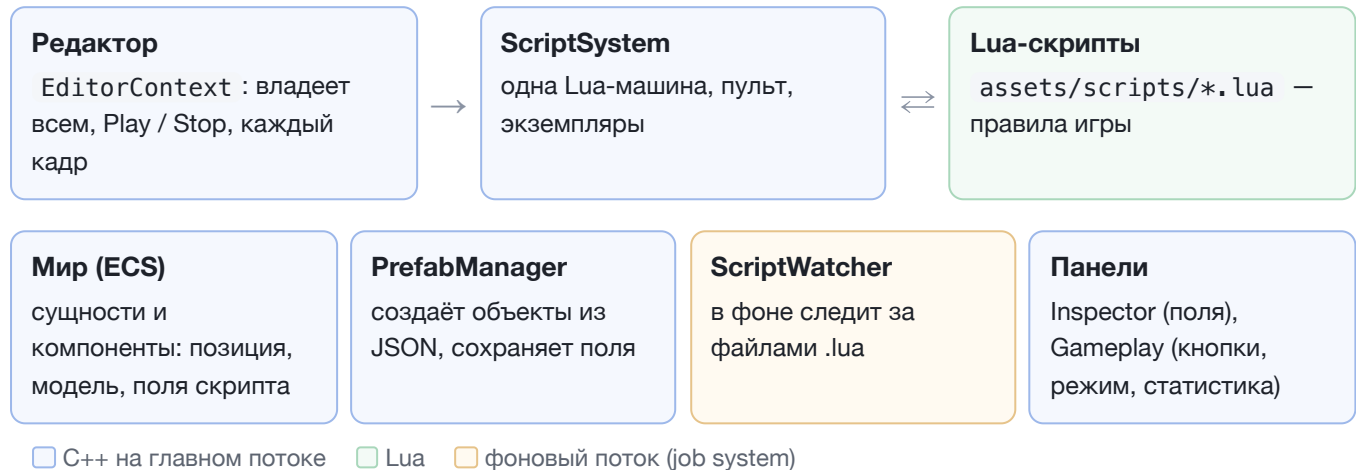
Режиссёр (`ScriptSystem`) читает сценарий и каждую секунду говорит актёрам, что делать. Если переписать страницу сценария посреди спектакля, режиссёр подхватит её без остановки. Это и есть hot reload.

Что требовало ТЗ (обязательная часть)

Пункт	Простыми словами	Где сделано
2.1	Встроить язык как часть движка: понятно, когда он создаётся и когда умирает	<code>ScriptSystem</code> — шаги 1–2
2.2	Движок зовёт скрипт, скрипт зовёт движок (минимум 4 группы функций), без опасных ссылок	биндинги, хэндлы, <code>LuaBehaviour</code> — шаги 3–5
2.3	Перезагрузить скрипты без перезапуска движка	кнопка Reload, а сверх ТЗ — автоматически и прямо в Play (допфичи 1–2)
2.4	Вся игровая механика на скриптах; правило можно поменять по просьбе преподавателя	<code>waves.lua</code> , <code>enemy.lua</code> , <code>core.lua</code> — шаг 11
2.5	Создание объектов по шаблону, числа поведения — в данных	JSON-префабы — шаг 9

Пункт	Простыми словами	Где сделано
2.6	Числа поведения правятся в редакторе и сохраняются	Inspector → Save to Prefab — шаг 10
3	Стабильность: сломанный скрипт не роняет движок, долгие сессии, корректный выход	шаг 12, лимиты, тесты

Из чего состоит система



Главное правило всей лабы

C++ ДАЁТ УСЛУГИ

- найти объект по тегу;
- создать объект из шаблона, удалить объект;
- прочитать и записать позицию;
- узнать, нажата ли клавиша;
- физика и рендер целиком.

LUA ПРИНИМАЕТ РЕШЕНИЯ

- когда начать волну и сколько врагов;
- куда идёт враг и сколько урона наносит;
- кого уничтожит импульс;
- когда поражение;
- что написать на экране.

Обязательная часть · как интеграция делалась по порядку

1 Положили Lua внутрь движка

Сначала нужно, чтобы движок вообще умел выполнять Lua-код.

ПРОСТЫМИ СЛОВАМИ

Lua — это обычная программа на языке C, около 15 тысяч строк. Мы скачали её исходники и **скомпилировали вместе с движком**, как ещё одну часть. В итоге Lua сидит прямо внутри файла `GameEngine`.

Пользователю ничего ставить не надо: ни Lua, ни DLL рядом с программой.

- 1 **Скачали исходники.** Скрипт `tools/setup_lua.sh` (для Mac/Linux) или `tools/setup_lua.ps1` (для Windows) скачивает Lua 5.4.8 и sol2 3.3.1 и сверяет контрольные суммы, чтобы файлы были ровно те. Исходники ложатся в `external/lua` и `external/sol2`.
- 2 **Собрали библиотеку.** `cmake/LuaRuntime.cmake` берёт все `.c`-файлы Lua, кроме консольных программ `lua.c` и `luac.c`, и собирает статическую библиотеку `EngineLua`. Если исходников нет, сборка сразу говорит, какой скрипт запустить.
- 3 **Подключили к движку и тестам.** В `CMakeLists.txt` движок и тесты линкуются с `EngineLua`.
- 4 **Включили проверки sol2.** Флаг `SOL_ALL_SAFEITIES_ON`: sol2 проверяет типы и не пропускает, например, строку туда, где ждут число.
- 5 **Маленькая заплатка.** В sol2 3.3.1 есть ошибка, на которой спотыкается новый компилятор clang. CMake правит копию одного файла в папке сборки, а скачанные исходники не трогает.

ГДЕ В КОДЕ

<code>cmake/LuaRuntime.cmake</code>	сборка Lua в библиотеку <code>EngineLua</code> , заплатка sol2
<code>tools/setup_lua.sh</code> , <code>tools/setup_lua.ps1</code>	скачивание исходников с проверкой хэша
<code>external/lua/lua-5.4.8</code> , <code>external/sol2/sol2-3.3.1</code>	сами исходники
<code>CMakeLists.txt</code>	движок и тесты подключают <code>EngineLua</code>

ЕСЛИ СПРОСЯТ

Что нужно поставить на компьютер игрока, чтобы скрипты работали?

Ничего. Lua скомпилирована прямо внутрь exe (статическая линковка), отдельного интерпретатора нет.

Почему Lua, а не Python?

Lua крошечная (~200 КБ), написана на чистом C, создана как раз для встраивания и быстро работает. Python весит ~10 МБ, и его пришлось бы класть рядом с программой.

Что такое sol2?

Библиотека для C++, которая сама переводит значения и функции между C++ и Lua. Без неё пришлось бы вручную класть каждое значение на стек Lua через C API.

2 Одна «Lua-машина» на весь редактор

Нужен один хозяин, который создаёт Lua, настраивает и в конце аккуратно закрывает.

ПРОСТЫМИ СЛОВАМИ

Хозяин — класс `ScriptSystem`. Внутри у него ровно одна Lua-машина (`sol::state`). Её создают при запуске редактора, а закрывают при выходе.

Play и Stop машину **не пересоздают**. Они только создают и удаляют «живые копии» поведения для объектов, то есть экземпляры.

Что происходит, когда `ScriptSystem` создаётся

- 1 **Создаётся «песочница»** (`Sandbox`): счётчик памяти и будильник на время. Подробно — в допфиче 3.
- 2 **Создаётся Lua-машина** с нашей функцией выдачи памяти: так мы видим каждую её покупку памяти.
- 3 **Открываются только 4 библиотеки**: `base`, `math`, `table`, `string`. Файлов (`io`), операционной системы (`os`) и загрузки модулей (`require`) нет: скрипт не может ничего испортить на диске.
- 4 **`print` переделан**: текст уходит не в терминал, а в консоль редактора, с файлом и строкой.
- 5 **`dofile` и `loadfile` убраны**: скрипт не читает файлы в обход движка.
- 6 **Ставятся обработчики ошибок** (шаг 12).
- 7 **Регистрируется модуль `uzlezz`** — «пульт» от движка (шаг 3).
- 8 В лог пишется `Lua runtime initialized`.

Порядок смерти важен

При выходе `~ScriptSystem` сначала вызывает `stop()` (все экземпляры получают `on_destroy`), потом закрывает Lua-машину, а уже потом умирает песочница. Это обеспечено порядком полей в `struct ScriptSystem::Impl`: в C++ поля уничтожаются в обратном порядке. Песочница объявлена первой, значит умирает последней, потому что Lua до самого конца отдаёт ей память. В логе видно `Lua runtime shut down`, затем `Shutdown complete`.

ГДЕ В КОДЕ

<code>src/scripting/ScriptSystem.h</code>	интерфейс: <code>start</code> , <code>stop</code> , <code>update</code> , <code>reload</code> , <code>hotReload</code> , <code>stats</code>
<code>src/scripting/ScriptSystem.cpp</code>	<code>struct ScriptSystem::Impl</code> и его конструктор <code>Impl(World&)</code> : вся настройка выше; функция <code>print</code> ; деструктор <code>~ScriptSystem</code>
<code>src/editor/EditorContext.h/.cpp</code>	редактор владеет полем <code>scripts</code> ; там же движок передаёт <code>ScriptSystem</code> две услуги: <code>spawnPrefab</code> (создать префаб) и <code>inputPressed</code> (нажата ли клавиша)

ЕСЛИ СПРОСЯТ

Сколько у вас Lua state и сколько он живёт?

Один, от создания редактора до выхода. Play/Stop пересоздают только экземпляры поведения, а классы и машина остаются.

Что такое «единая точка загрузки»?

Все файлы скриптов выполняются через одну функцию `Impl::run`, и только на главном потоке.

3 Пульт от движка для скрипта

Скрипт сам ничего не умеет: ему нужно выдать функции движка. Это и называется биндинги.

ПРОСТЫМИ СЛОВАМИ

Мы даём скрипту «пульт» с кнопками. Каждая кнопка — маленькая функция на C++ (лямбда). `sol2` превращает её в функцию, которую Lua может вызвать.

Скрипт пишет `self.entity:get_position()`, а на самом деле выполняется C++-код, который достаёт позицию из компонента `Transform`.

Все кнопки пульта (4 группы, как требует T3)

Группа	Функция в Lua	Что делает
1. Сущность и позиция	<code>is_alive()</code> , <code>id</code>	жив ли объект, его номер
	<code>get_position()</code> / <code>set_position(v)</code>	прочитать и записать позицию
	<code>set_yaw(угол)</code> , <code>set_animation_speed(x)</code>	повернуть, ускорить анимацию
	<code>get_tag()</code>	тег объекта («Core», «Enemy»)

Группа	Функция в Lua	Что делает
	<code>get_field(имя) / set_field(имя, x)</code>	поля поведения из инспектора
2. Ввод	<code>world:input_pressed("StartWave")</code>	нажата ли клавиша этого действия в этом кадре
3. Создать / удалить	<code>world:spawn_prefab(путь, позиция)</code>	создать объект из JSON-шаблона
	<code>world:destroy(e)</code>	удалить (после кадра, см. шаг 8)
4. Поиск в мире	<code>world:find_by_tag(тер)</code>	первый живой объект с тегом
	<code>world:find_all_by_tag(тер)</code>	все объекты с тегом, обычный Lua-массив с 1
Прочее	<code>world:set_status(текст) , print(...)</code>	строка на экране игры; печать в консоль

Ещё в модуле есть `uzlezz.Vec3(x, y, z)` — вектор. Он передаётся **копией**: чтобы сдвинуть объект, нужно вызвать `set_position` с новым вектором.

Защита внутри кнопок

- Каждая кнопка сначала проверяет хэндл (`h.check()`), шаг 4).
- В позицию нельзя записать `NaN` или бесконечность: будет ошибка `Position must be finite`.
- `input_pressed` отвечает «да», только если окно Game в фокусе. Поэтому на демо надо **кликнуть по окну Game**, иначе Space и F до скрипта не дойдут.
- `find_*` не видят объекты, уже помеченные на удаление.

ГДЕ В КОДЕ

[src/scripting/ScriptSystem.cpp](#)

функция `bindEngine`: все типы (`new_usertype`) `ScriptBehaviour`, `Vec3`, `Entity`, `World` и функция `uzlezz.behaviour`

[src/editor/EditorContext.cpp](#)

клавиши: `bindAction("StartWave", Space)`, `bindAction("DefensePulse", F)`; фокус окна Game → `allowGameInput`

Номерок вместо куртки (хэндлы)

Самый частый вопрос на защите: как скрипт ссылается на объекты движка и что будет, если объект удалили.

ПРОСТЫМИ СЛОВАМИ

В гардеробе тебе дают не куртку, а **номерок**. Если куртку уже забрали, по номерку тебе просто скажут «такой нет». Ничего страшного не случится.

Так же и скрипт: он держит не указатель на объект в памяти, а номерок — **хэндл**.

Указатель опасен: враг удалён, память освобождена, а указатель всё ещё смотрит туда.

Обращение по нему — мусор или падение программы.

Из чего состоит номерок

```
struct EntityHandle {
    World* world;           // в каком мире
    std::weak_ptr<int> lifetime; // «талон» мира: протухает при очистке мира
    Entity id;              // номер сущности
    std::uint64_t generation; // поколение: какая по счёту жизнь у этого номера
};
```

Три проверки перед каждым действием (`valid()`)

- 1 **Мир тот же?** «Талон» мира не протух. Он протухает, когда мир очищают, например на Stop.
- 2 **Сущность жива?** Мир говорит, что такой номер есть.
- 3 **Поколение совпало?** Номер мог освободиться и достаться новому объекту. Поколение отличает «старого врага №5» от «нового врага №5».

Если все три «да», действие выполняется. Если нет: `is_alive()` вернёт `false`, а любая другая кнопка даст ошибку `Stale EntityHandle` («протухший хэндл») с файлом и строкой. Ошибка выключит только этот экземпляр, движок продолжит работу.

Что случилось	<code>is_alive()</code>	<code>get_position()</code>
Сохранили врага №5 в <code>self.target</code>	<code>true</code>	позиция
Враг дошёл до ядра и удалён	<code>false</code>	ошибка <code>Stale EntityHandle</code>
Нажали Stop: сцена восстановлена, враг №5 снова есть	<code>false</code>	ошибка: талон мира сменился, поколение новое

То же для мира: `self.world`

`self.world` хранит **слабую ссылку** (`weak_ptr`) на состояние игры `ApiState`. При Stop создаётся новый `ApiState`, и старая ссылка «протухает». Если кто-то сохранил старый `self.world`, он получит ошибку `Script world is no longer active`.

ГДЕ В КОДЕ

`src/scripting/ScriptSystem.cpp`

`struct EntityHandle ( valid , check )`, функция `handle()` , `struct ApiState` , `struct WorldApi`

`src/ecs/World.h`

`generation(e)` — поколение; `lifetime()` — «талон» мира, меняется при `clear()`

ЕСЛИ СПРОСЯТ

Почему не `shared_ptr` ?

Тогда объект не смог бы умереть, пока скрипт его держит: «осиротевший» объект, утечка. Хэндл ничем не владеет, он только проверяет.

А C++ держит Lua-объекты?

Только внутри `ScriptSystem::Impl` (таблицы классов и экземпляров). Они уничтожаются раньше Lua-машины, это гарантирует порядок полей.

5

C++-база и Lua-наследник

T3 требует, чтобы класс поведения в скрипте «наследовал» класс из C++, как в лекции 5 с `pybind11`.

ПРОСТЫМИ СЛОВАМИ

В C++ есть класс-«розетка» `ScriptBehaviour` с тремя методами: `on_create` , `on_update` , `on_destroy` . Движок умеет звонить только в эти методы.

Мы сделали наследника `LuaBehaviour` — **переадресацию звонка**. Движок звонит в обычный C++-метод `on_update` , а `LuaBehaviour` переводит звонок в Lua-функцию `Enemy:on_update` . Такой приём называется **trampoline** («батут»).

Как это выглядит в Lua

```
Enemy = uzlezz.behaviour {
    fields = {speed = 1.5, damage = 1}
}
function Enemy:on_update(dt)
    local p = self.entity:get_position()
    ...
end
```

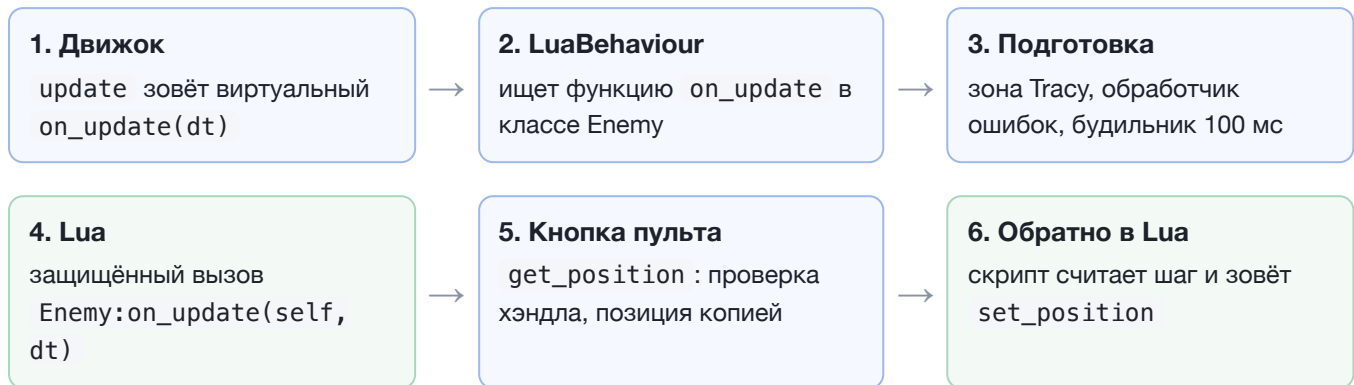
-- объявили класс от C++-базы
-- числа для инспектора
-- движок зовёт каждый кадр
-- кнопка пульта

`uzlezz.behaviour` — три строчки Lua внутри движка. Он делает две вещи:

- записывает в класс отметку `base = uzlezz.ScriptBehaviour` . По ней движок проверяет, что класс правильный;

- подвешивает метатаблицу с `__index` на C++-базу. Это и есть «наследование» в Lua: чего нет в классе, ищется в базе.

Путь одного вызова



Если Lua-функция упала, `LuaBehaviour` превращает ошибку в C++-исключение с текстом. Цикл кадра его ловит и выключает только этот экземпляр.

Где ищется метод

Каждый экземпляр — своя таблица `self` с `self.entity`, `self.world` и своими переменными. Если в `self` чего-то нет, Lua идёт по цепочке: `self` → класс `Enemy` → C++-база. Тонкость: движок берёт метод **только из класса** (`raw_get`). Если класс не объявил `on_destroy`, движок его просто не вызывает.

ГДЕ В КОДЕ

`src/scripting/ScriptSystem.h`

`class ScriptBehaviour` — C++-база с виртуальными методами

`src/scripting/ScriptSystem.cpp`

`class LuaBehaviour` и его метод `call(slot, name)`; в конце `bindEngine` — Lua-функция `uzlezz.behaviour`; `Impl::makeObject` — таблица экземпляра

ЕСЛИ СПРОСЯТ

Как это связано с `pybind11` из лекции?

Один в один. `PyBehaviourTrampoline` + `PYBIND11_OVERRIDE` у нас — `LuaBehaviour` + `call`. Python-класс наследует `gr.PyBehaviour`, а у нас Lua-класс объявлен через `uzlezz.behaviour`.

Как файл .lua становится классом

Объект в мире знает только «путь к файлу» и «имя класса». Как из этого получается работающее поведение?

ПРОСТЫМИ СЛОВАМИ

У каждого объекта со скриптом есть карточка `ScriptComponent` : «мой скрипт — `assets/scripts/enemy.lua` , класс — `Enemy` , вот мои числа». `ScriptSystem` выполняет этот файл, достаёт из него таблицу `Enemy` , проверяет её и запоминает.

```
struct ScriptComponent {
    std::string path;           // "assets/scripts/enemy.lua"
    std::string className;     // "Enemy"
    std::string prefab;        // из какого JSON создан (для Save to Prefab)
    std::map<std::string, ScriptValue> fields; // speed = 1.5, damage = 1 ...
};
```

Загрузка по шагам

- 1 `run` выполняет файл в **своём отдельном окружении** (environment). Это как отдельная папка для переменных каждого файла: `enemy.lua` и `core.lua` не перетирают глобальные переменные друг друга.
- 2 `extract` достаёт из окружения таблицу с именем класса и проверяет:
 - она есть;
 - объявлена через `uzlezz.behaviour` (отметка `base`);
 - есть таблица `fields` с допустимыми значениями (bool, целое, число, строка);
 - `on_create` , `on_update` , `on_destroy` , `on_reload` — функции или их нет.
- 3 Класс кладётся в «шкаф» `classes` по ключу "путь:Класс" , например `"assets/scripts/enemy.lua:Enemy"` . Каждый класс загружается один раз.

Кнопка Reload Scripts в Edit: «всё или ничего»

`ScriptSystem::reload` сначала загружает **все** файлы сцены в отдельный, временный шкаф. Если хоть один файл с ошибкой, временный шкаф выбрасывается, и работают старые классы. Если всё хорошо, шкафы меняются местами (`swap`). Такое поведение называется **транзакционным**.

ГДЕ В КОДЕ

<code>src/scripting/ScriptComponent.h</code>	карточка скрипта у сущности
<code>src/scripting/ScriptSystem.cpp</code>	<code>Impl::run</code> (выполнить файл или текст), <code>Impl::extract</code> (проверить класс), <code>Impl::load</code> , <code>Impl::key</code> , <code>ScriptSystem::reload</code>
<code>src/editor/EditorContext.cpp</code>	<code>reloadScripts</code> — кнопка Reload Scripts; <code>loadArenaScene</code> — после загрузки арены вызывает <code>scripts.reload()</code>
<code>src/editor/panels/GameplayPanel.cpp</code>	сама кнопка Reload Scripts

ЕСЛИ СПРОСЯТ

Как движок узнаёт, какой класс в файле главный?

Не угадывает: имя класса записано в префабе (`"class": "Enemy"`). Движок только проверяет, что класс правильный.

Что если в файле синтаксическая ошибка?

Загрузка падает с текстом «файл:строка: ...», старые классы остаются, игра идёт дальше.

7 Что происходит на Play и Stop

Play запускает игру, Stop возвращает сцену как было. Посмотрим, что при этом делают скрипты.

Нажали Play (⌘P)

- 1 Снимок сцены.** Редактор запоминает все объекты и компоненты (`captureSnapshot`), чтобы Stop мог всё вернуть.
- 2 Превью врага убирается.** Это объект только для настройки полей; в игре враги создаются из JSON.
- 3 `scripts.start()` :**
 - перечитывает все файлы с диска (`reload`);
 - включает пульт мира (`api->active = true`), подключает услуги «создать префаб» и «нажата клавиша»;
 - для каждой сущности со скриптом создаёт **экземпляр** (`create`).
- 4 Создание экземпляра (`Impl::create`):**
 - поля карточки приводятся к объявлению класса (`defaults` , шаг 10);
 - создаётся таблица `self` с `self.entity` (хэндл) и `self.world` (пульт мира);
 - создаётся C++-объект `LuaBehaviour` (батут);
 - вызывается `on_create` ;

- запоминается снимок `self.*` после `on_create`: он понадобится для hot reload L2.

- 5 Если хоть что-то упало, Play **отменяется**, сцена возвращается, в окне Gameplay видна ошибка.

Нажали Stop

- 1 **Пульт мира выключается** (`api->active = false`): в `on_destroy` уже нельзя спавнить.
- 2 **У каждого экземпляра вызывается `on_destroy`**, экземпляры удаляются.
- 3 **Создаётся новый `ApiState`**: все старые `self.world` навсегда протухают.
- 4 **Полная сборка мусора Lua**: таблицы экземпляров уходят из памяти.
- 5 **Сцена восстанавливается из снимка** (`restoreSnapshot`): мир очищается (талон меняется, старые хэндлы протухают), объекты возвращаются как до Play.

ПРОСТЫМИ СЛОВАМИ

Play — актёры выходят на сцену и получают свои роли (экземпляры). Stop — актёры уходят, декорации ставят как было. А театр, то есть Lua-машина и загруженные классы, остаётся.

ГДЕ В КОДЕ

<code>src/editor/EditorContext.cpp</code>	<code>play()</code> , <code>stop()</code> , <code>captureSnapshot()</code> , <code>restoreSnapshot()</code>
<code>src/scripting/ScriptSystem.cpp</code>	<code>ScriptSystem::start</code> , <code>ScriptSystem::stop</code> , <code>Impl::create</code> , <code>Impl::makeObject</code> , <code>Impl::snapshot</code> , <code>Impl::remove</code>

8 Один кадр игры

Каждый кадр в Play редактор вызывает `scripts.update(dt, allowInput)`, до физики и анимации.

- 1 **Копия списка объектов.** Перебираем копию, а не сам список. Если скрипт в этом кадре создаст врага, перебор не сломается: новый враг получит экземпляр и `on_create` в следующем кадре.
- 2 **У каждого объекта со скриптом — `on_update(dt)`**. Если экземпляра ещё нет (объект только что заспаунили), он сначала создаётся.
- 3 **`destroy` ничего не удаляет сразу**, а только кладёт объект в список «на удаление». Удалить объект посреди перебора — всё равно что выдернуть стул из-под человека: следующий скрипт мог бы получить объект без компонентов.
- 4 **Ошибка в скрипте** ловится в цикле. Текст уходит в лог, экземпляр помечается `failed` и больше не вызывается. Остальные работают, игра идёт.

- 5 **После перебора** — настоящее удаление: `on_destroy` , потом `destroyEntity` .
- 6 **Хвост кадра:** маленький шаг сборщика мусора Lua (`lua_gc(LUA_GCSTEP, 32)`), замер времени скриптов для окна Gameplay, графики для Tracy.

ПРОСТЫМИ СЛОВАМИ

Учитель проверяет тетради по списку класса, напечатанному утром. Новичок, пришедший днём, попадёт в список завтра. Двоечника, которого «отчислили» посреди проверки, вычеркнут после проверки, а не во время.

Пример кадра

Объект Что произошло в этом кадре

Core `on_update` : перезарядка импульса уменьшилась

Waves `on_update` : таймер вышел → `spawn_prefab` → враг №7 появился в мире, но не в списке кадра

Enemy №5 дошёл до ядра: HP ядра 10 → 9, `destroy(self)` → помечен; после перебора удалён

Enemy №6 ошибка в скрипте → `failed` , больше не тикает, остальные живут

Enemy №7 получит экземпляр и `on_create` в следующем кадре

ГДЕ В КОДЕ

[src/scripting/ScriptSystem.cpp](#)

`ScriptSystem::update` — весь кадр скриптов; в `bindEngine` кнопка `destroy` (только пометка в `ApiState::destroyed`)

[src/editor/EditorContext.cpp](#)

`updateGameplay` — первым делом `scripts.update` , потом физика; `allowGameInput` = окно Game в фокусе

ЕСЛИ СПРОСЯТ

Почему удаление отложенное?

Чтобы не ломать перебор и порядок: это «очередь изменений» из лекции 5.

Модель обновления у вас какая (из лекции)?

Batched: одна система обходит все скрипты разом. И скрипты — отдельная фаза кадра до физики.

Префабы — шаблоны в JSON

Чтобы волна могла создать врага одной командой, враг должен быть описан шаблоном.

ПРОСТЫМИ СЛОВАМИ

Префаб — это **формочка для печенья**. В файле JSON написано, из чего состоит объект: тег, позиция, модель, анимация, скрипт и его числа. Команда `spawn_prefab` «выдавливает» по формочке новый объект.

```
// assets/prefabs/enemy.json (сокращённо)
{ "components": {
  "tag": "Enemy",
  "meshRenderer": { "mesh": "assets/models/animation/Walking.fbx" },
  "animator": { "speed": 1.0 },
  "script": { "path": "assets/scripts/enemy.lua", "class": "Enemy",
    "fields": { "speed": 1.5, "damage": 1, "attack_radius": 1.2 } } } }
```

Как работает создание (`PrefabManager::spawn`)

- 1 Читает JSON и разбирает компоненты: позицию, скрипт с полями, анимацию, коллайдер, физику, модель.
- 2 Проверяет данные: вектор из 3 чисел, числа конечные, поля только bool, целое, число или строка.
- 3 Создаёт сущность и добавляет компоненты. Модель и текстура грузятся **асинхронно** через job system из ЛР 1.
- 4 Если что-то сломалось посередине, **полусозданная сущность удаляется** (откат), и ошибка идёт дальше. В мире не остаётся «половинок».

Кто создаёт объекты

- **Загрузка арены** (кнопка Open Arena или флаг `--editor-arena`): `loadArenaScene` создаёт `arena.json`, `core.json`, `waves.json` и превью `enemy.json`.
- **Скрипт волн в игре**: `self.world:spawn_prefab("assets/prefabs/enemy.json", позиция)`. Вызов идёт через услугу `spawnPrefab`, которую редактор передал в `ScriptSystem`.

ГДЕ В КОДЕ

<code>src/prefabs/PrefabManager.cpp</code>	<code>PrefabManager::spawn</code> — создание с откатом
<code>assets/prefabs/*.json</code>	<code>arena</code> , <code>core</code> , <code>waves</code> , <code>enemy</code>
<code>src/editor/EditorContext.cpp</code>	<code>loadArenaScene</code> ; строка <code>scripts.spawnPrefab = ...</code> — услуга для Lua
<code>src/scripting/ScriptSystem.cpp</code>	кнопка <code>spawn_prefab</code> в <code>bindEngine</code>
<code>src/core/LaunchOptions.cpp</code>	флаг <code>--editor-arena</code> — запуск сразу в арену

10 Поля и инспектор

Числа баланса (скорость, урон, HP) должны быть видны и редактируемы в редакторе, а правка — сохраняться в префаб.

ПРОСТЫМИ СЛОВАМИ

Значение поля собирается из **трёх слоёв**, как бутерброд. Верхний слой побеждает:

1. **Lua** (`fields = {speed = 1.5}`) — какой тип у поля и значение по умолчанию;
2. **JSON префаба** (`"speed": 2.0`) — переопределение для этого шаблона;
3. **Инспектор** (в Edit) или `set_field` из скрипта (в Play) — правка прямо сейчас.

Итоговое значение хранится в карточке `ScriptComponent::fields` . Скрипт читает его через `get_field` при каждом вызове, поэтому видит правки сразу.

Тонкость с числами: 2 и 2.0

В Lua 5.4 `2` — целое, а `2.0` — дробное. Функция `coerce` решает, что можно:

Ситуация	Результат
В JSON <code>"speed": 2</code> , а в Lua <code>speed = 1.0</code>	можно: <code>2</code> → <code>2.0</code> , тихо
<code>set_field("health", 4.0)</code> , а health целое	можно: дробной части нет → <code>4</code>
<code>set_field("health", 8.5)</code> , а health целое	ошибка <code>field 'health' expects integer, got number</code> , значение не меняется
Поле удалили из Lua, а в JSON оно осталось	поле выбрасывается, в лог <code>[WARN] ... not declared ... dropped</code>
В Lua сменили тип поля	значение сбрасывается к умолчанию, <code>[WARN] ... reset to default</code>

Ситуация	Результат
В Lua добавили новое поле	появляется со значением по умолчанию, даже прямо в Play после hot reload

Эту «уборку» делает функция `defaults`. Благодаря ей правка списка полей не ломает reload.

Инспектор и Save to Prefab

- 1 Выделили объект: в карточке **Script** видны файл, префаб и все поля. Под каждый тип свой виджет: галочка, целое, число, строка.
- 2 В Edit правка пишется прямо в `ScriptComponent`. В Play поля только для чтения, но обновляются вживую (видно, как падает HP ядра).
- 3 **Save to Prefab** (`PrefabManager::saveFields`):
 - читает JSON;
 - проверяет, что путь и класс скрипта в файле не поменялись;
 - заменяет `fields`, сохраняя порядок ключей (`ordered_json`), поэтому `git diff` маленький;
 - пишет сначала во временный `.tmp`, потом переименовывает. Если программа упадёт посреди записи, старый файл останется целым.
- 4 **Reset** в карточке сбрасывает поля к значениям из Lua и работает только в Edit.

ГДЕ В КОДЕ

<code>src/scripting/ScriptSystem.cpp</code>	<code>coerce</code> , <code>Impl::defaults</code> , кнопки <code>get_field</code> / <code>set_field</code> , <code>fromLua</code> / <code>toLua</code> (перевод значений)
<code>src/editor/panels/InspectorPanel.cpp</code>	<code>InspectorPanel::drawScript</code> — карточка Script, кнопки Save to Prefab и Reset
<code>src/prefabs/PrefabManager.cpp</code>	<code>PrefabManager::saveFields</code>
<code>src/editor/EditorContext.cpp</code>	<code>savePrefabFields</code> — вызов сохранения и сообщение

ЛОВУШКА ДЕМО

Правка поля у **превью врага** без Save to Prefab в новых врагах не появится: в игре они создаются из JSON на диске. Сначала Save, потом Play.

11 Сама игра на Lua

Механика «защити ядро от волн» целиком лежит в трёх файлах. В C++ нет ни одного правила игры.

Файл	Что делает	Поля (числа баланса)
waves.lua класс Waves	Space запускает волну, но только если прошлая уничтожена. Волна $N = 4 + 2 \cdot (N-1)$ врагов. Враги появляются по одному каждые 0,6 с на окружности радиуса 10 вокруг ядра, через равные углы. Пишет строку HUD «Core HP · Wave · Alive · Pending». При HP = 0 — DEFEAT.	first_wave_count = 4 , count_increment = 2 , spawn_interval = 0.6 , spawn_radius = 10.0 , enemy_prefab
enemy.lua класс Enemy	Идёт к ядру и поворачивается к нему. Когда расстояние $\leq 1,2$, снимает HP ядра и удаляет себя. Если ядро мертво, замирает.	speed = 1.5 , damage = 1 (целое!), attack_radius = 1.2 , animation_speed , target_tag = "Core"
core.lua класс Core	Хранит HP. По F, если перезарядка прошла, уничтожает всех врагов в радиусе 6 (импульс), затем перезарядка 2 с.	health = 10 , pulse_radius = 6.0 , pulse_cooldown = 2.0

Как живёт состояние

- **Числа баланса** — это поля (`get_field`). Их видно в инспекторе и префабе.
- **Текущее состояние игры** — переменные экземпляра `self.*`: номер волны `self.wave`, сколько ещё спавнить `self.remaining`, таймер `self.timer`, перезарядка ядра `self.cooldown`.
- HP ядра — поле `health`. Враг уменьшает его через `core:set_field("health", ...)`, и инспектор видит это вживую.

Как поменять правило по просьбе преподавателя

Просьба	Что поменять
Число: враги вдвое быстрее	enemy.lua : <code>get_field("speed") * 3 * dt</code>
Число: каждая волна быстрее	waves.lua : после спавна <code>enemy:set_field("speed", 1.5 + 0.5 * (self.wave - 1))</code>
Условие: импульс только при ≥ 3 врагах в радиусе	core.lua : сначала собрать врагов в радиусе в список, уничтожать, только если <code>#hit >= 3</code>
Порядок: заход против часовой	waves.lua : в формуле угла <code>+ self.serial</code> заменить на <code>- self.serial</code>

Сохранил файл (⌘S) — изменение работает сразу, даже в Play (допфики 1–2). Готовые куски кода лежат в `docs/lab2/DEMO.md`.

ГДЕ В КОДЕ

`assets/scripts/waves.lua`, `enemy.lua`, `core.lua` вся механика

`assets/prefabs/waves.json`, `enemy.json`, `core.json` значения полей для этих объектов

`docs/lab2/DEMO.md` сценарий демо и готовые правки А–Г

ЛОВУШКА ДЕМО

`damage` — **целое**. Если написать `damage = 1.5` или поделить через `/` (в Lua `/` всегда даёт дробное), будет ошибка типа. Для целого деления используй `//`.

12 Ошибки не роняют движок

По ТЗ падение движка из-за ошибки в скрипте отнимает весь блок «Стабильность» (4 балла). Поэтому ошибки ловятся на каждом уровне.

ПРОСТЫМИ СЛОВАМИ

Любой вызов Lua идёт «в перчатках» — это **защищённый вызов** (protected call). Если внутри скрипта что-то сломалось, ошибка не вылетает наружу и не роняет программу. Она возвращается как результат «не получилось, вот текст». Мы пишем текст в лог и выключаем только виноватый экземпляр.

Чтобы по ошибке было понятно, где она

- **traceback** — к сообщению ошибки добавляется стек Lua: кто кого вызвал.
- **locatedException** — ошибки из наших C++-кнопок (`Stale EntityHandle`, неверный тип поля, неизвестное поле) получают приписку «файл:строка» того места в скрипте, откуда кнопку нажали. Без этого sol2 отдавал бы только текст исключения без места.
- Опечатка в имени поля даёт понятное `Enemy: unknown field 'helth'`.
- В окно Gameplay идёт первая строка ошибки, в лог (Console) — полный текст со стеком.

Что происходит при разных поломках

Поломка	Реакция движка	Что видно
Синтаксическая ошибка, сохранили файл	новый код не применяется, работает старый	<code>enemy.lua:16: 'end' expected</code>

Поломка	Реакция движка	Что видно
Ошибка во время игры	выключен один экземпляр; оживёт после исправления файла	<code>enemy.lua:15: Enemy: unknown field 'helth'</code>
Бесконечный цикл	прерван через 100 мс (допфича 3)	<code>core.lua:16: script exceeded the 100 ms time budget</code>
Скрипт съедает память	отказ в памяти, экземпляр выключен (допфича 3)	<code>Hog:on_update: Lua heap limit exceeded (256 MB)</code>
Ошибка в <code>on_create</code> при Play	Play отменяется, сцена как была	«Play cancelled: ...»
Выход из редактора	<code>stop</code> → закрытие Lua → песочница	<code>Lua runtime shut down , Shutdown complete</code>

ГДЕ В КОДЕ

<code>src/scripting/ScriptSystem.cpp</code>	<code>traceback</code> , <code>locatedException</code> , <code>fieldOf</code> (unknown field), <code>Impl::report</code> (первая строка → UI, всё → лог), <code>LuaBehaviour::call</code> (защищённый вызов), <code>ScriptSystem::update</code> (флаг failed)
<code>src/editor/panels/GameplayPanel.cpp</code>	показ ошибки и числа выключенных экземпляров
<code>src/editor/panels/ConsolePanel.cpp</code>	консоль редактора, куда идут лог и <code>print</code>

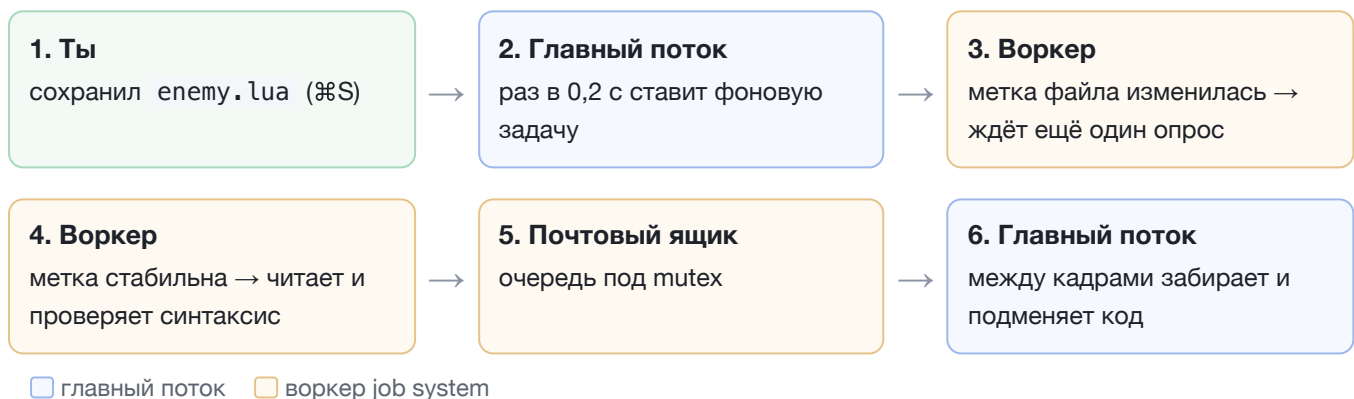
1 Движок сам замечает сохранение

По ТЗ хватило бы кнопки «перезагрузить». Мы сделали так, что движок сам видит сохранённый файл, причём проверка идёт в фоне через job system из ЛР 1.

ПРОСТЫМИ СЛОВАМИ

Начальник (главный поток) занят: он рисует кадры. Поэтому раз в 0,2 секунды он говорит помощнику (воркеру): «сходи проверь, не поменялись ли файлы». Помощник обходит папку, находит изменённый файл, читает его и проверяет, без ошибок ли он написан. Результат кладёт в **почтовый ящик**. Начальник между кадрами заглядывает в ящик и применяет изменения.

Кадр не ждёт ни диска, ни проверки. Применяет код только главный поток, потому что Lua-машина не умеет работать из нескольких потоков сразу.



Подробнее о каждом шаге

- Опрос.** Каждый кадр редактор спрашивает вотчер: «пора?» (`due`). Пора, если прошло 0,2 с и прошлая задача закончилась. Тогда `poll` отдаёт задачу в job system через `submitBackground` с низким приоритетом. Главный поток эту задачу «не крадёт», поэтому кадр её не ждёт.
- Обход (`scan`).** Воркер собирает все `.lua` из `assets/scripts` и файлы, которые использует сцена. У каждого файла снимает **метку**: время последней записи плюс размер.
- Защита от «половины файла» (`debounce`).** Редактор кода иногда пишет файл в два приёма. Если проверить его посередине, будет ложная ошибка. Поэтому изменённая метка сначала попадает в «ожидание». Файл берётся в работу, только когда метка одинакова два опроса подряд.
- Проверка (`validate`).** Файл читается в память и **только компилируется** в отдельной одноразовой Lua-машине (`luaL_newstate` + `luaL_loadbufferx`). Код при этом не выполняется. Ошибка будет в обычном виде «путь:строка: текст».
- Почтовый ящик.** Результат `{путь, текст файла, ok, ошибка}` кладётся в очередь `ready` под `mutex` (замок, чтобы два потока не писали одновременно).

- 6 **Применение.** Главный поток забирает почту (`drain`) и для каждого изменения вызывает `applyScriptChange` . Если синтаксис плохой, в лог и окно Gameplay пишется «Not reloaded, the previous code stays active», и игра идёт на старом коде. Если хороший, вызывается `scripts.hotReload(...)` с уже прочитанным текстом (допфича 2). Файл второй раз не читается, так что гонки с редактором кода нет.

Если сохранили файл, который сцене не нужен (например, `notes.lua`), вотчер его проверит, но `hotReload` найдёт ноль классов и ничего не сделает.

ГДЕ В КОДЕ

<code>src/scripting/ScriptWatcher.h</code>	класс вотчера, <code>interval = 0.2</code> , <code>due</code> , структура <code>Change</code>
<code>src/scripting/ScriptWatcher.cpp</code>	<code>poll</code> (поставить задачу), <code>scan</code> (обход, метки, <code>debounce</code>), <code>validate</code> (компиляция), <code>drain</code> (забрать почту), деструктор ждёт задачу
<code>src/jobs/JobSystem.cpp</code>	<code>submitBackground</code> — фоновая задача (из ЛП 1, <code>enkiTS</code>)
<code>src/editor/EditorContext.cpp</code>	в <code>update</code> : <code>scriptWatcher.poll(...)</code> и цикл по <code>scriptWatcher.drain()</code> ; <code>applyScriptChange</code>
<code>src/editor/panels/GameplayPanel.cpp</code>	галочка «Reload on save»

ЕСЛИ СПРОСЯТ

Почему не проверять файлы прямо на главном потоке?

Обход папки, чтение и компиляция — это диск и CPU. Кадр бы подтормаживал. Правило семестра из ЛП 1: тяжёлое — задачей в пул, итог — на главном потоке.

Почему не применить код прямо на воркере?

Lua-машина однопоточная: если трогать её из двух потоков, данные испортятся. Поэтому воркер только проверяет текст в своей отдельной одноразовой машине.

2 Подмена кода прямо в игре (L1 / L2)

Код меняется прямо в Play, без Stop. Номер волны и HP при этом можно сохранить.

ПРОСТЫМИ СЛОВАМИ

Идёт третья волна. Ты меняешь в `enemy.lua` скорость и сохраняешь. Каждый враг получает «нового себя» на новом коде. Дальше два варианта:

- **L1 (Reset)** — новый объект начинает с нуля, как после `on_create` ;
- **L2 (Keep state)** — в новый объект переносятся старые переменные `self.*` : номер волны, таймеры и т. д. Как переезд в новую квартиру со своими вещами.

Как работает `hotReload(путь, текст, режим)`

- 1 **Найти, кого касается файл:** какие классы из него загружены и какие объекты сцены его используют. Если никого — выйти.
- 2 **Собрать и проверить новые классы** (`run` + `extract`). Если ошибка — **ничего не трогаем:** старые классы и экземпляры работают дальше.
- 3 **Подменить классы** в «шкафу» и привести поля карточек к новому объявлению (`defaults`): новые поля появятся, удалённые уйдут.
- 4 **Если идёт Play**, для каждого живого экземпляра этого файла:
 - создать новый объект на новом классе и вызвать `on_create` ;
 - L1 — так и оставить;
 - L2 — перенести состояние (`transfer`). Если класс сам объявил `on_reload(old)` , переносит он;
 - если новый `on_create` упал, экземпляр остаётся на старом коде.
- 5 Экземпляры, выключенные ошибкой, **оживают** на исправленном коде (revived). Сломал, исправил — игра продолжилась.
- 6 В лог: `waves.lua reloaded in Play: 1 instance(s), state kept (L2): 5 field(s)` .

Правило переноса в L2 (самое хитрое место)

Переменная `self.x` переносится из старого объекта в новый, если выполнены все три условия:

- 1 такая переменная есть и в новом объекте (после нового `on_create`);
- 2 тип тот же (число и число, таблица и таблица);
- 3 **её начальное значение в коде не поменяли.** Для этого при создании каждого экземпляра мы запоминаем снимок `self.*` сразу после `on_create` (`initial`) и сравниваем старый снимок с новым.

`self.entity` и `self.world` никогда не копируются: у нового объекта они свои.

Что поменяли в <code>waves.lua</code>	Что будет в L2
Ничего в <code>on_create</code> , только логику	<code>wave = 3</code> , <code>remaining = 2</code> , <code>timer</code> , <code>serial</code> , <code>wave_count</code> перенесены — 5 полей
В <code>on_create</code> : <code>self.timer = 1.0</code> вместо <code>0.0</code>	<code>timer = 1,0</code> : раз начальное значение поменяли, значит хотели новое
Добавили <code>self.bonus = 5</code>	<code>bonus = 5</code> : нового поля раньше не было
Переименовали <code>serial</code> в <code>spawned</code>	<code>serial</code> выброшено, <code>spawned = 0</code>
Объявили <code>on_reload(old)</code>	переносится только то, что класс сам скопировал

Режим L1/L2 выбирается в окне Gameplay: «Keep state (L2)» / «Reset state (L1)». По умолчанию L2.

ГДЕ В КОДЕ

<code>src/scripting/ScriptSystem.h</code>	<code>enum class ReloadMode {KeepState, Reset}, struct HotReloadReport</code>
<code>src/scripting/ScriptSystem.cpp</code>	<code>ScriptSystem::hotReload, Impl::transfer, Impl::snapshot, поле Instance::initial, LuaBehaviour::on_reload</code>
<code>src/editor/EditorContext.cpp</code>	<code>applyScriptChange</code> — вызов и текст сообщения; <code>reloadScripts</code> в Play идёт тем же путём
<code>src/editor/panels/GameplayPanel.cpp</code>	выбор режима Keep state / Reset

3 Лимиты: время и память (sandbox)

Защита от скрипта, который завис в бесконечном цикле или съедает всю память.

Время: будильник на 100 мс

ПРОСТЫМИ СЛОВАМИ

Перед каждым вызовом скрипта мы ставим **будильник** на 100 мс. Lua через каждые 1000 своих команд звонит нам (это «хук») и спрашивает: «можно дальше?». Если будильник уже прозвенел, мы отвечаем ошибкой. Зависший скрипт останавливается, а редактор продолжает работать.

- `lua_sethook(L, budgetHook, LUA_MASKCOUNT, 1000)` — хук на каждые 1000 инструкций, ставится один раз в конструкторе.
- `Deadline` — будильник. Ставится в `LuaBehaviour::call` и при загрузке файла, а снимается сам при выходе из функции (RAII). Если внутри `on_update` заспавнили врага и вызвался его `on_create`, вложенный будильник **не продлевает** внешний.
- `budgetHook` сверяет часы с дедлайном. Если время вышло — ошибка `core.lua:16: script exceeded the 100 ms time budget` со строкой, где крутится цикл.
- Обычный `on_update` — примерно 0,02 мс и пара сотен инструкций. Хук почти ничего не стоит.

Память: потолок 256 МБ

ПРОСТЫМИ СЛОВАМИ

Lua не берёт память сама: она **просит** её через функцию-«кассира», которую мы ей дали. Кассир ведёт счёт. Если покупка вывела бы за 256 МБ, кассир отказывает. Lua пробует собрать мусор и попросить снова. При повторном отказе получается ошибка памяти, и выключается только жадный экземпляр.

- `allocate` — наш аллокатор. Lua-машина создана через `sol::state(panic, &allocate, &sandbox)`.
- Считаем занято (`used`) и пик (`peak`). Они видны в окне Gameplay: строка «Lua heap».
- Освобождение и уменьшение блока не отказываем никогда: этого требует Lua.
- При нехватке памяти Lua не зовёт обработчик ошибок. Поэтому текст собирает сам `LuaBehaviour::Hog::on_update: Lua heap limit exceeded (256 MB)`.

И сама песочница

Нет `io`, `os`, `require`, `debug`, `dofile`, `loadfile`: скрипт не может читать или удалять файлы и запускать программы (шаг 2).

ГДЕ В КОДЕ

<code>src/scripting/ScriptSystem.cpp</code>	<code>struct Sandbox, allocate, budgetHook, class Deadline; lua_sethook</code> в конструкторе <code>Impl</code> ; проверка <code>call_status::memory</code> в <code>LuaBehaviour::call</code>
<code>src/scripting/ScriptSystem.h</code>	<code>struct ScriptLimits (100 мс, 256 МБ), setLimits, stats</code>
<code>tests/ScriptRuntimeTests.cpp</code>	<code>runawayScriptIsInterrupted, heapLimitIsEnforced</code>

ЕСЛИ СПРОСЯТ

Почему хук каждые 1000 инструкций, а не на каждую?

На каждую — слишком дорого. 1000 инструкций выполняются за микросекунды, так что точность всё равно отличная.

4 Скрипты в профайлере Tracy

Трасу (из лекции 2) показывает, сколько времени занимает каждая часть кадра. Мы добавили туда скрипты, причём поимённо.

ПРОСТЫМИ СЛОВАМИ

Зона Tracy — как секундомер с подписью: «вот этот кусок кода шёл столько-то». Мы поставили секундомеры на все важные места скриптинга. На каждый вызов скрипта секундомер подписан именем класса, поэтому в Tracy видно отдельно, сколько стоит тик `Enemy`, а сколько `Waves`.

Что видно в Tracy

Зоны `Lua: update`, `Lua: start`,
`Lua: reload`, `Lua: hot reload`,
`Lua: load file`

Где поставлено

`ZoneScopedN` в одноимённых функциях `ScriptSystem.cpp`

Что видно в Trasy

Где поставлено

Зона на каждый вызов:

`Enemy:on_update` ,
`Waves:on_update` , `Core:on_update`

`LuaBehaviour::call : ZoneScopedN("Lua callback") +`
`ZoneName(имя)` , имена заготовлены в конструкторе

Зоны вотчера `Lua watcher: scan` и
`Lua watcher: validate` — на
дорожке **воркера**, а не главного
потока

`ScriptWatcher.cpp : scan , validate`

Графики `Lua memory KB` и `Lua`
`instances`

`TracyPlot` в конце `ScriptSystem::update`

Без Trasy те же цифры видны в окне **Gameplay**: «Lua heap» (занято / пик / потолок), «Script time» (мс за кадр), число выключенных ошибкой экземпляров. Их отдаёт `ScriptSystem::stats()` .

Как показать: задача VS Code «**Engine + Tracy**». В Trasy через Find zone по имени

`Enemy:on_update` видно среднее время тика врага. График памяти должен быть ровным: это доказательство, что память не утекает.

ГДЕ В КОДЕ

`src/scripting/ScriptSystem.cpp`

все `ZoneScopedN` , `ZoneName` в `LuaBehaviour::call` ,
`TracyPlot` в `update` , `stats()`

`src/scripting/ScriptWatcher.cpp`

зоны на воркере

`src/editor/panels/GameplayPanel.cpp`

строки `Lua heap` и `Script time`

`.vscode/tasks.json`

задачи «Engine + Tracy», «Tracy: connect»

5 Подсказки в VS Code

Пишешь `self.entity:` , и VS Code показывает список функций движка с типами.

ПРОСТЫМИ СЛОВАМИ

VS Code не знает, какие функции есть у нашего движка: они живут внутри C++. Поэтому мы написали для него «**справочник**» — файл с описанием всех функций на языке специальных комментариев (`---@class` , `---@param` , `---@return`). Движок этот файл никогда не запускает, его читает только расширение Lua в VS Code.

- 1 **Справочник** `tools/lua-stubs/uzlezz.lua` описывает `uzlezz.Vec3` , `uzlezz.Entity` , `uzlezz.World` , `uzlezz.Behaviour` и функцию `uzlezz.behaviour` , с типами параметров и коротким описанием.
- 2 **Настройка** `.luarc.json` : версия Lua 5.4, подключить папку `tools/lua-stubs` , выключить подсказки для `io` / `os` / `debug` (их у нас нет).

- 3 **Рекомендация расширения** `.vscode/extensions.json` : VS Code сам предложит поставить `sumneko.lua` (`lua-language-server`).
- 4 **Пометки в скриптах:** `---@class Enemy : uzlezz.Behaviour`. Теперь VS Code знает, что у `self` есть `entity` и `world`.
- 5 **Тест, что справочник не врёт.** `ScriptSystem::apiNames()` читает настоящий список функций прямо из Lua-машины (метатаблицы типов). Тест `ideStubsMatchBindings` сверяет его со справочником **в обе стороны**: если добавили функцию в движок и забыли описать (или описали несуществующую), `ctest` падает.

ГДЕ В КОДЕ

<code>tools/lua-stubs/uzlezz.lua</code>	справочник API
<code>.luarc.json</code>	настройка <code>lua-language-server</code>
<code>.vscode/extensions.json</code>	рекомендация расширения <code>sumneko.lua</code>
<code>assets/scripts/*.lua</code>	пометки <code>---@class ... : uzlezz.Behaviour</code>
<code>src/scripting/ScriptSystem.cpp</code>	<code>apiNames()</code> и <code>collectApi</code> — список реальных биндингов
<code>tests/ScriptRuntimeTests.cpp</code>	<code>ideStubsMatchBindings</code>

Что ещё подтянули в обязательной части

Это не допфичи, а мелкие доработки, без которых демо могло сорваться:

- Ошибки из C++-кнопок теперь с «файл:строка» (`locatedException`), а опечатка в имени поля даёт понятный текст.
- `print` из Lua попадает в консоль редактора; `dofile` / `loadfile` закрыты.
- Удалённое из Lua поле больше не ломает `reload` (`defaults`).
- `Save to Prefab` сохраняет порядок ключей JSON (`ordered_json`).
- `Reset` полей отключён в `Play`; ошибка Lua сбрасывается при смене сцены.
- Скрипт установки Lua для Mac (`tools/setup_lua.sh`) и флаг запуска сразу в арену (`--editor-arena`).
- Задача VS Code «Engine: run» запускает движок из корня репозитория. Иначе правки в `assets/` были бы не видны: движок читал бы копию в `build/`.

19 · Тесты: чем доказано, что всё работает

Команда `ctest --test-dir build/mac --output-on-failure` запускает 10 наборов тестов, все зелёные. Два набора — про скрипты.

`tests/ScriptSystemTests.cpp` — механика и долгая жизнь

- Арена грузится, волны идут, враги доходят и снимают HP: проверка механики на копиях префабов, поэтому правка баланса на демо не ломает тест.
- **100 циклов** Stop → Reload → Play: после каждого Stop ноль экземпляров, ничего не копится.
- **30 минут** симулированной игры: пик памяти Lua после первой минуты почти не растёт (порог +4 МБ).

`tests/ScriptRuntimeTests.cpp` — 8 сценариев допфич и ошибок

Тест	Что проверяет
<code>bindingErrorsHaveLocation</code>	ошибки из C++-кнопок содержат «файл:строка»
<code>printGoesToLogAndFilesAreClosed</code>	<code>print</code> идёт в лог с файлом и строкой, <code>dofile / loadfile</code> закрыты
<code>fieldTypesAreCoerced</code>	2 ↔ 2.0, дробное в целое — ошибка, удалённое поле не ломает reload
<code>runawayScriptIsInterrupted</code>	<code>while true do end</code> прерван по бюджету, остальные экземпляры работают
<code>heapLimitIsEnforced</code>	огромная строка упирается в потолок памяти
<code>hotReloadInPlay</code>	L1 и L2, <code>on_reload</code> , сломанный файл не ломает игру, оживление экземпляров, 200 reload подряд без роста памяти
<code>watcherUsesJobSystem</code>	вотчер с запущенным job system замечает правку временного файла и отклоняет синтаксическую ошибку
<code>ideStubsMatchBindings</code>	справочник для VS Code совпадает с реальными биндингами

ГДЕ В КОДЕ

`tests/ScriptSystemTests.cpp`,
`tests/ScriptRuntimeTests.cpp`

сами тесты

`CMakeLists.txt`

цели `ScriptSystemTests` и `ScriptRuntimeTests`,
`add_test`

20 • Три сценария целиком

Если голова путается, пройди эти три истории по порядку: в них собраны все шаги сразу.

История 1. Нажали Play, потом Space

- 1 `EditorContext::play` запоминает сцену и вызывает `scripts.start()`.
- 2 `start` перечитывает `core.lua`, `waves.lua`, `enemy.lua`, проверяет классы, включает пульт мира.
- 3 У Core и Waves создаются экземпляры, вызываются `on_create`: `self.cooldown = 0`, `self.wave = 0` ...
- 4 Каждый кадр `updateGameplay` → `scripts.update` → `on_update` у Core и Waves через `LuaBehaviour`.
- 5 Кликнули по окну Game и нажали Space. `input_pressed("StartWave")` вернул `true`, и Waves начал волну 1 из 4 врагов.
- 6 Раз в 0,6 с `spawn_prefab("assets/prefabs/enemy.json", точка на круге)` → `PrefabManager::spawn` создаёт врага.
- 7 В следующем кадре у врага появляется экземпляр `Enemy`, он идёт к ядру, дошёл — `set_field("health", 9)` и `destroy`.

История 2. Во время игры поменяли скорость в `enemy.lua` и сохранили

- 1 Через $\leq 0,2$ с главный поток ставит задачу вотчера в job system.
- 2 Воркер видит новую метку `enemy.lua` → «ожидание»; на следующем опросе метка та же → читает и компилирует. Синтаксис верен → в почтовый ящик.
- 3 Главный поток: `drain` → `applyScriptChange` → `hotReload("assets/scripts/enemy.lua", текст, L2)`.
- 4 Новый класс `Enemy` проверен и подменён. Каждый живой враг получает новый объект, `on_create`, перенос `self.*`.
- 5 Враги сразу идут быстрее. Номер волны и HP не сбросились: они в Waves и в поле ядра, а этот файл их не трогал.

История 3. Сохранили `enemy.lua` с ошибкой

- 1 Воркер компилирует файл и получает `assets/scripts/enemy.lua:31: 'end' expected ...`. В ящик уходит «ok = false».
- 2 Главный поток: в лог ошибка, в окне Gameplay — «Not reloaded, the previous code stays active».
- 3 Враги продолжают идти на **старом** коде, ничего не упало.
- 4 Исправили и сохранили — через ~0,4 с новый код применён.

- 5 Если ошибка была не в синтаксисе, а во время выполнения (например, `get_field("helth")`), враги с ней выключились (`failed`). После исправления они **оживают**: в сообщении будет «revived after an error».

21 • Вопросы на защите: короткие ответы

Отвечают двое, кого назовёт преподаватель; за каждый ответ 0, 1 или 2 балла. Ниже — самое вероятное.

1. Как устроено встраивание Lua?

Lua 5.4 и sol2 собраны внутри exe. Один `sol::state` в `ScriptSystem`, которым владеет редактор. Он живёт от запуска до выхода, а Play/Stop пересоздают только экземпляры.

2. Как движок вызывает скрипт?

У C++-базы `ScriptBehaviour` виртуальные `on_create/on_update/on_destroy`. Наследник `LuaBehaviour` (trampoline) пересылает вызов в функцию Lua-класса защищённым вызовом.

3. Как скрипт вызывает движок? Сколько категорий API?

Через модуль `uzlezz` (биндинги sol2), 4 категории: сущность и трансформ, ввод, спавн и удаление, поиск по тегу. Плюс статус HUD и print.

4. Как скрипт ссылается на объекты? Что если объект удалён?

Через хэндл: id + поколение + талон мира, не указатель. Удалён — `is_alive()` вернёт false, остальное даст ошибку «Stale EntityHandle» с файлом и строкой. Движок жив.

5. Что с ссылками после Stop?

Мир очищается, талон меняется — все хэндлы протухают. `self.world` тоже протухает: при Stop создаётся новый `ApiState`.

6. Как работает hot reload?

Вотчер раз в 0,2 с ставит задачу в job system. Воркер находит изменённый файл (метка стабильна два опроса), компилирует его в одноразовом state и кладёт в очередь. Главный поток забирает и вызывает `hotReload`: транзакционно, при ошибке остаётся старый код.

7. Что такое L1 и L2?

L1 — после reload экземпляр начинает с нуля. L2 — переносится `self.*`, кроме полей, чьё начальное значение поменяли в коде. Класс может сам перенести всё в `on_reload(old)`.

8. Почему удаление отложенное?

Чтобы не ломать перебор объектов в кадре. `destroy` помечает, удаление идёт после обхода.

9. Что будет, если скрипт завис или жрёт память?

Хук каждые 1000 инструкций проверяет бюджет 100 мс и прерывает вызов ошибкой. Аллокатор не даёт больше 256 МБ. Выключается только этот экземпляр.

10. Где хранятся числа баланса и как их править?

Тип и умолчание — в `fields` Lua-класса, переопределение — в JSON префаба, итог — в `ScriptComponent`. Правятся в инспекторе, Save to Prefab пишет в JSON.

11. Почему поля в компоненте, а не в Lua?

Компонент — простые данные: его копирует снимок Play, видит инспектор, пишет JSON. Это не зависит от жизни Lua-машины.

12. Где у вас лекция 5 «спектр скриптинга»?

Уровни 3–4: скрипт объявляет новые типы поведения и компоненты на скрипте. Физика, рендер и анимация остаются в C++.

13. Почему вотчер на job system, а не на главном потоке?

Диск и компиляция не должны тормозить кадр. А применять код можно только на главном потоке: Lua-машина однопоточная.

14. Как вы доказали, что нет утечек?

Тесты: 30 минут симуляции, 100 Stop/Play, 200 reload подряд — память не растёт. Вживую — ровный график «Lua memory KB» в Tracy.

22 · Шпаргалка: что где лежит

Файл	Что там
src/scripting/ScriptSystem.h	интерфейс подсистемы, ScriptBehaviour , ReloadMode , лимиты
src/scripting/ScriptSystem.cpp	почти всё : хэндлы, песочница, LuaBehaviour , bindEngine , загрузка, Play/Stop, кадр, hot reload, перенос L2
src/scripting/ScriptWatcher.h/.cpp	вотчер файлов на job system
src/scripting/ScriptComponent.h	карточка скрипта: путь, класс, префаб, поля
src/prefabs/PrefabManager.cpp	создание из JSON с откатом, Save to Prefab
src/editor/EditorContext.cpp	владелец всего: Play/Stop, кадр, арена, клавиши, применение reload
src/editor/panels/InspectorPanel.cpp	карточка Script, Save to Prefab, Reset
src/editor/panels/GameplayPanel.cpp	Open Arena, Reload Scripts, Reload on save, L1/L2, Lua heap, Script time
src/ecs/World.h	поколения и «талон» мира для хэндлов
src/jobs/JobSystem.cpp	submitBackground из ЛР 1
assets/scripts/*.lua	механика: waves, enemy, core
assets/prefabs/*.json	шаблоны: arena, core, waves, enemy
tools/lua-stubs/uzlezz.lua , .luarc.json	подсказки в VS Code
cmake/LuaRuntime.cmake , tools/setup_lua.sh	Lua внутри exe
tests/ScriptSystemTests.cpp , tests/ScriptRuntimeTests.cpp	тесты
docs/lab2/DEM0.md , lab2- defense.html , lab2-defense.md	сценарий демо, интерактивная презентация, текстовая версия с вопросами

Главные цифры

Версии	Lua 5.4.8, sol2 3.3.1
Вотчер	опрос раз в 0,2 с; файл берётся, когда метка одинакова два опроса подряд
Лимиты	100 мс на вызов (хук каждые 1000 инструкций), 256 МБ куча
Ядро	10 НР, импульс радиус 6, перезарядка 2 с

Враг	скорость 1,5, урон 1 (целое), атака с 1,2
Волны	4 + 2 за каждую следующую, раз в 0,6 с, радиус 10
Клавиши	Space — волна, F — импульс, ⌘P — Play (сначала клик по окну Game!)
Тесты	10 наборов ctest; 30 мин симуляции, 100 Stop/Play, 200 reload

Запуск перед защитой

```

./build/mac/GameEngine --editor-arena      # сразу в арену (из корня репозитория!)
ctest --test-dir build/mac --output-on-failure # все тесты
git checkout assets/scripts assets/prefabs  # вернуть файлы после демо

```